

Signals and Systems

A Review of Mathematical Modeling, State Machines, and Linear Systems

Shashwat Saxena

IIIT Vadodara

June 5, 2026

Presentation Outline

- 1 Signals Overview
- 2 System Dynamics
- 3 State Spaces
- 4 Linear Systems
- 5 Applications & MATLAB

Defining Signals

Signals are mediums that convey information by mapping independent physical quantities onto dependent physical quantities.

Continuous-Time Signals

- Defined for every value of time within an interval.
- Modeled as: $x : \mathbb{R} \rightarrow \mathbb{R}$
- Examples: Audio pressure waves, temperature, electrical voltages.

Discrete-Time Signals

- Defined only at specific instants of time (using a discrete index n).
- Modeled as: $x : \mathbb{Z} \rightarrow \mathbb{R}$
- Examples: Digital images, sampled audio, data sequences.

Real-World Signal Mappings

- **Audio Signals:** Map time onto the relative change in air pressure. The average of an audio signal is zero.

$$\text{audio} : \text{time} \rightarrow \text{relativeChangeInAirPressure}$$

- **Images:** Map a 2D space onto light intensity. For greyscale, the co-domain is 1D.

$$\text{imageGreyscale} : [0, \text{height}] \times [0, \text{width}] \rightarrow [0, B_{\max}]$$

(Where $B_{\max}$ represents maximum intensity/white).

- **Videos & Sequences:** Videos map frames over time (frame rate). Sequences map natural numbers to entities (Data or Event Streams).

Systems: Memoryless and Dynamic

Systems act on input signals to produce output signals.

Memoryless Systems

- Output depends *only* on the present value of the input.
- Example:

$$y(t) = 2x(t)$$

Dynamic Systems

- Output depends on previous inputs or previous outputs (requires memory).
- Example:

$$y(n) = x(n) + x(n-1)$$

Continuous-Time: Differential Equations

- Used for physical systems (RC circuits, thermal systems).
- General first-order system:

$$\frac{dy(t)}{dt} + ay(t) = bx(t)$$

Discrete-Time: Difference Equations

- Used for digital filters and signal processing.
- First-order difference equation:

$$y(n+1) = ay(n) + bx(n)$$

State Spaces and State Machines

State Space: The minimum information required to completely determine a system's future behavior (e.g., the current arrangement of a Rubik's Cube).

Finite State Machines (FSM)

- Formally represented as a five-tuple:

(State, Input, Output, Update, InitialState)

- Example (Traffic Light):** Transitions between {Red, Yellow, Green} based on timing and conditions.
- Determinism:** Deterministic (one valid next state) vs. Non-Deterministic (multiple possible states).

System Composition and Behaviors

Complex systems are built by combining simpler state machines:

- **Synchronous:** All components evolve simultaneously on a common clock.
- **Parallel:** Subsystems operate side-by-side (distributed systems).
- **Cascade:** Output of $System_1$ is the input to $System_2$.

Feedback and Relations

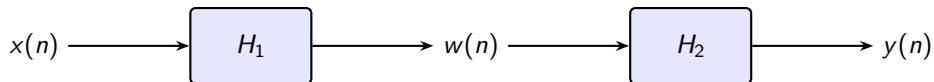
- **Feedback:** Outputs influence future inputs. Fixed points satisfy

$$f(x) = x$$

- **Simulation vs Bisimulation:** Simulation implies A reproduces B. Bisimulation implies behavioral equivalence (mutual simulation).

Cascade (Series) Composition

Idea: Output of one system feeds directly into the next.



Mathematically:

$$w(n) = H_1\{x(n)\}, \quad y(n) = H_2\{w(n)\} = H_2\{H_1\{x(n)\}\}$$

Key Property (LTI systems only)

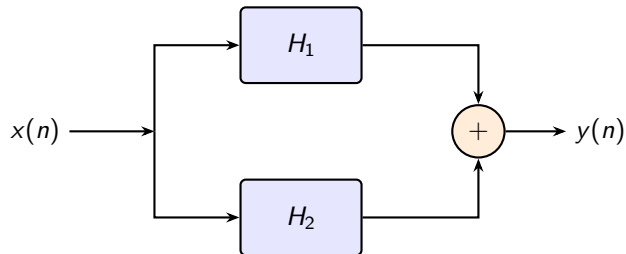
For LTI systems, cascade order does not matter: $H_1 \circ H_2 = H_2 \circ H_1$

In terms of impulse responses: $h(n) = h_1(n) * h_2(n)$

Example: An anti-aliasing filter followed by a decimator.

Parallel Composition

Idea: Same input goes to both systems; outputs are summed.



Mathematically:

$$y(n) = H_1\{x(n)\} + H_2\{x(n)\}$$

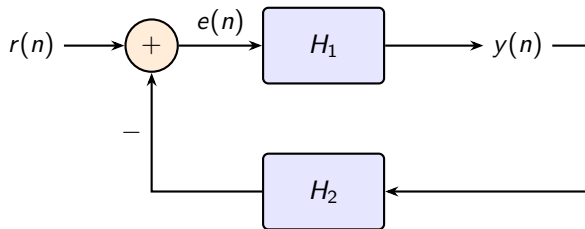
Key Property (LTI systems only)

Equivalent to a single system with impulse response: $h(n) = h_1(n) + h_2(n)$

Example: A bank of filters covering different frequency bands

Feedback Composition

Idea: Output is routed back and subtracted from (or added to) the input.



Mathematically:

$$\begin{aligned} e(n) &= r(n) - H_2\{y(n)\}, & y(n) &= H_1\{e(n)\} \\ \Rightarrow y(n) &= \frac{H_1}{1 + H_1 H_2} \{r(n)\} \quad (\text{closed-loop}) \end{aligned}$$

Why feedback?

Introduces memory, enables self-regulation, and can stabilize unstable systems — at the cost of added complexity

Linear Time-Invariant (LTI) Systems

A system is linear if it satisfies the superposition principle, which consists of two properties:

- **Homogeneity:**

$$f(ax) = af(x)$$

- **Additivity:**

$$f(u + v) = f(u) + f(v)$$

Combined Linearity property for arbitrary scalars a and b :

$$f(au + bv) = af(u) + bf(v)$$

Time Invariance

- Behavior does not change over time. Delaying the input delays the output identically:

$$x(n - n_0) \rightarrow y(n - n_0)$$

One-Dimensional State-Space Systems

A discrete-time SISO system with a single state variable $s(n)$ stores memory:

$$s(n+1) = as(n) + bx(n)$$

$$y(n) = cs(n) + dx(n)$$

Moving Average Example

$$y(n) = \frac{x(n) + x(n-1)}{2}$$

By choosing $s(n) = x(n-1)$, the state-space realization becomes:

$$s(n+1) = x(n)$$

$$y(n) = \frac{s(n)}{2} + \frac{x(n)}{2}$$

Linear State-Space: The Setup

For a linear system, the state, input, and output are **vectors** real vector spaces.

The three spaces:

- State vector: $\mathbf{s}(n) \in \mathbb{R}^p$
- Input vector: $\mathbf{x}(n) \in \mathbb{R}^m$
- Output vector: $\mathbf{y}(n) \in \mathbb{R}^q$

Each component of $\mathbf{s}(n)$ stores *one piece of memory* about the system's past.

Example (2nd-order system):

$$\mathbf{s}(n) = \begin{bmatrix} s_1(n) \\ s_2(n) \end{bmatrix}, \quad \mathbf{x}(n) = [x_1(n)]$$

s_1 might store the current position, s_2 the velocity
— both needed to predict future motion.

The Update and Output Maps

Since the system is **linear**, the Update and Output functions must be linear maps — represented by matrices.

State Update Equation

$$\mathbf{s}(n+1) = A\mathbf{s}(n) + B\mathbf{x}(n)$$

Output Equation

$$\mathbf{y}(n) = C\mathbf{s}(n) + D\mathbf{x}(n)$$

Matrix dimensions (must be consistent for multiplication):

- $A \in \mathbb{R}^{p \times p}$ — maps state to next state
- $B \in \mathbb{R}^{p \times m}$ — maps input to state
- $C \in \mathbb{R}^{q \times p}$ — maps state to output
- $D \in \mathbb{R}^{q \times m}$ — maps input directly to output (feedthrough)

Concrete Example: Moving Average as a State-Space System

Consider the 2-tap moving average:

$$y(n) = \frac{1}{2}x(n) + \frac{1}{2}x(n-1)$$

Choose state $s(n) = x(n-1)$ (the one past sample we need). Then $p = m = q = 1$, so all matrices are scalars:

$$\underbrace{A}_0 s(n) + \underbrace{B}_1 x(n) \Rightarrow s(n+1) = x(n)$$
$$y(n) = \underbrace{C}_{\frac{1}{2}} s(n) + \underbrace{D}_{\frac{1}{2}} x(n)$$

Intuition: $A = 0$ means the state doesn't feed back into itself — no dynamics, just memory. Generalizing to larger A matrices gives richer system behavior (oscillation, decay, instability).

Initial State: $\mathbf{s}(0) = \mathbf{0}$ (system starts at rest) — this completes the five-tuple.

SISO vs MIMO Systems

SISO (Single-Input Single-Output) systems have one input signal and one output signal.

- **Audio amplifier:** One microphone signal $x(t)$ is amplified into one speaker signal $y(t)$.
- **RC low-pass filter:** One voltage input produces one filtered voltage output.

MIMO (Multiple-Input Multiple-Output) systems have multiple input and output signals, often modeled with vectors:

$$\mathbf{y}(t) = H\mathbf{x}(t)$$

- **Wireless communication:** Multiple transmit antennas send signals to multiple receive antennas.
- **Car control system:** Steering and acceleration inputs affect position, speed, and direction outputs.

Several fundamental behaviors that were simulated in MATLAB:

- **Basic Signals:** Generated and plotted discrete unit impulse, square pulse, unit ramp, and exponential decay in matlab.
- **State Machines:** Experimenting with different Linear Time Invariant State Machines, like oscillators, moving average, low pass filters, etc.
- **Communication Channel Simulator:**
 - Built a software modulator combining a 50Hz base signal with a 220Hz carrier wave.
 - Visualized Amplitude Modulation (AM) demonstrating signal preparation for transmission.

Basic Signals

basicSignals.png

The Discrete-Time Oscillator

Goal: Construct a system whose state traces a circle in $\mathbb{R}^2$ indefinitely — a discrete-time harmonic oscillator.

Key Insight: Rotation in 2D is a *linear map*. If we apply the same rotation repeatedly, the state vector traces a circle.

Autonomous Linear State-Space System

$$\mathbf{s}(n+1) = A\mathbf{s}(n)$$

No input. The entire behavior is determined by:

- The matrix A (system dynamics)
- The initial state $\mathbf{s}(0)$ (initial condition)

The solution is simply:

$$\mathbf{s}(n) = A^n \mathbf{s}(0)$$

Stability depends entirely on the eigenvalues of A .

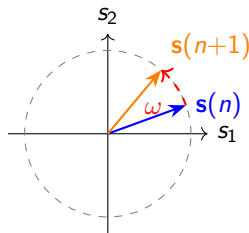
The Rotation Matrix

A rotation by angle ω (in radians) in $\mathbb{R}^2$ is given by:

$$A = \begin{bmatrix} \cos \omega & -\sin \omega \\ \sin \omega & \cos \omega \end{bmatrix}$$

Properties of A :

- $\det(A) = \cos^2 \omega + \sin^2 \omega = 1$
- A is **orthogonal**: $A^{-1} = A^T$
- Eigenvalues: $\lambda = e^{\pm j\omega}$, so $|\lambda| = 1$
- Since $|\lambda| = 1$: system is **marginally stable** — neither growing nor decaying



In MATLAB: $w = 0.01$ rad/step $\Rightarrow$ one full rotation every $\frac{2\pi}{0.01} \approx 628$ steps.

State-Space Realization

The oscillator state vector $\mathbf{s}(n) \in \mathbb{R}^2$ stores both coordinates:

$$\mathbf{s}(n) = \begin{bmatrix} s_1(n) \\ s_2(n) \end{bmatrix} = \begin{bmatrix} \cos(n\omega) \\ \sin(n\omega) \end{bmatrix} \quad \text{for } \mathbf{s}(0) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Update equation expanded:

$$\begin{bmatrix} s_1(n+1) \\ s_2(n+1) \end{bmatrix} = \begin{bmatrix} \cos \omega & -\sin \omega \\ \sin \omega & \cos \omega \end{bmatrix} \begin{bmatrix} s_1(n) \\ s_2(n) \end{bmatrix}$$

Which gives:

$$s_1(n+1) = s_1(n) \cos \omega - s_2(n) \sin \omega$$

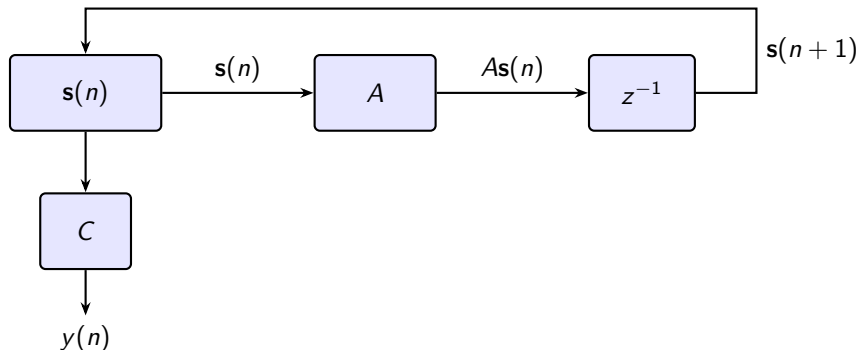
$$s_2(n+1) = s_1(n) \sin \omega + s_2(n) \cos \omega$$

Output

Extracting the first component via $C = \begin{bmatrix} 1 & 0 \end{bmatrix}$:

$$y(n) = C \mathbf{s}(n) = s_1(n) = \cos(n\omega)$$

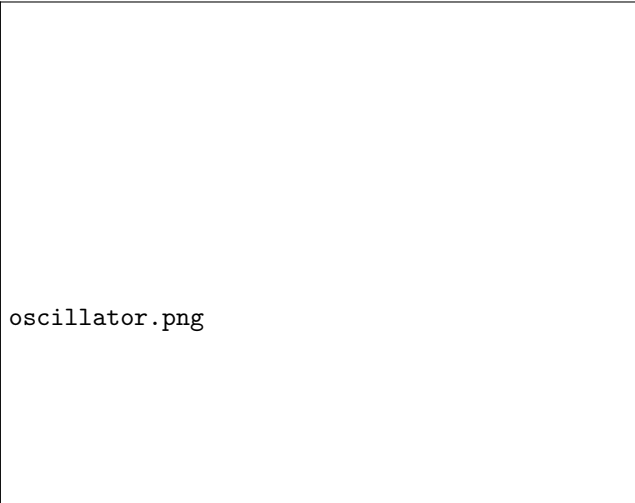
Block Diagram of the Oscillator



- z^{-1} is the **unit delay operator**: stores the current state for the next step
- The feedback loop through A is what sustains the oscillation
- No input $x(n)$ — purely autonomous dynamics

MATLAB Simulation: Phase Portrait

Initial state: $\mathbf{s}(0) = [0 \quad 1]^\top$, $\omega = \pi/60$ rad/step, $n = 1000$ steps



oscillator.png

The Moving Average Filter

Problem: A clean signal is corrupted by high-frequency noise. How do we recover it?

Idea: Average the current sample with the previous one. High-frequency oscillations cancel out; the slow-moving signal survives.

The 2-Tap Moving Average

$$y(n) = \frac{x(n) + x(n-1)}{2}$$

- $x(n)$ — current input sample
- $x(n-1)$ — one past sample (stored in state)
- Requires **exactly one unit of memory**

In our MATLAB model: a 5 Hz base signal corrupted by 440 Hz noise at half amplitude:

$$x(n) = \sin(2\pi \cdot 5 \cdot t_n) + 0.5 \sin(2\pi \cdot 440 \cdot t_n)$$

State-Space Realization

The filter has one state variable: $s(n) = x(n - 1)$, the previous input.

Update and output equations:

$$s(n + 1) = A s(n) + B x(n)$$

$$y(n) = C s(n) + D x(n)$$

Substituting $s(n) = x(n - 1)$:

$$y(n) = \frac{1}{2}x(n - 1) + \frac{1}{2}x(n)$$

ABCD Parameters

$$A = 0 \quad B = 1 \quad C = \frac{1}{2} \quad D = \frac{1}{2}$$

- $A = 0$: state does **not** feed back into itself — no internal dynamics
- $B = 1$: input is stored in state directly
- $C = D = \frac{1}{2}$: output is the mean of current and previous input

Stability: The Role of A

For a scalar state-space system, the zero-input response is:

$$s(n) = A^n s(0)$$

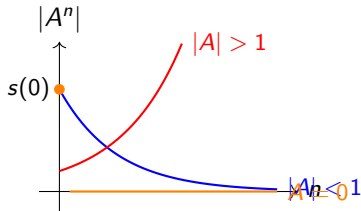
Stability condition:

- $|A| < 1 \Rightarrow$ response decays $\rightarrow$ **stable**
- $|A| = 1 \Rightarrow$ response constant $\rightarrow$ **marginally stable**
- $|A| > 1 \Rightarrow$ response grows $\rightarrow$ **unstable**

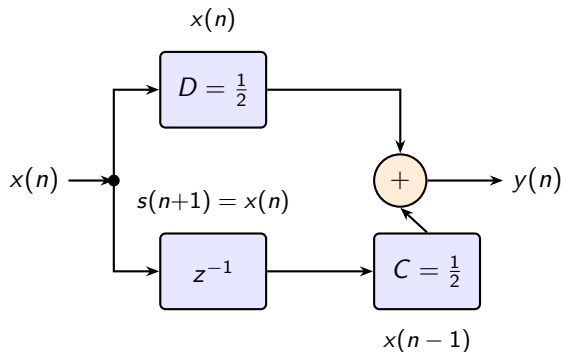
For our filter, $A = 0$:

$$s(n) = 0^n \cdot s(0) = 0 \quad \forall n \geq 1$$

The state has **no autonomous dynamics** — it is purely driven by the input.



Block Diagram of the Moving Average Filter



- z^{-1} : unit delay — stores $x(n]$, outputs $x(n-1]$ on the next step
- No feedback path from output to input — this is an **FIR** (Finite Impulse Response) filter
- $A = 0$ confirms this: state never feeds back into itself

Why Does It Remove High-Frequency Noise?

Apply input $x(n) = e^{j\omega n}$ (complex exponential) to find the frequency response:

$$y(n) = \frac{1}{2}e^{j\omega n} + \frac{1}{2}e^{j\omega(n-1)} = e^{j\omega n} \cdot \frac{1}{2}(1 + e^{-j\omega})$$

The **frequency response** $H(e^{j\omega})$ is:

$$H(e^{j\omega}) = \frac{1}{2}(1 + e^{-j\omega})$$

Its magnitude:

$$|H(e^{j\omega})| = \left| \cos \frac{\omega}{2} \right|$$

Interpretation

- $\omega = 0$ (DC): $|H| = 1$ — passes through unchanged
- $\omega = \pi$ (Nyquist, highest freq): $|H| = 0$ — completely rejected
- 440 Hz noise is near $\omega = \pi$ at $F_s = 1000$ Hz $\Rightarrow$ strongly attenuated

MATLAB Simulation Result

Setup: $F_s = 1000$ Hz, base signal = $\sin(2\pi \cdot 5 \cdot t)$, noise = $0.5 \sin(2\pi \cdot 440 \cdot t)$

movingAverageLowPassFilter.png